

Unicode Encoding Conversions for the Standard Library

Proposes Unicode Transformation Form (UTF) encoding conversion functions to ease interoperability between the strings of `char`, `char16_t`, `char32_t`, and `wchar_t` character types. Pure addition to the standard library. No changes to the core language or existing standard library components. Breaks no existing code or ABI. Specified in accordance with the Unicode Standard. Proposed wording provided. Has been implemented. Suitable for either a library TS or the standard itself.

This is a preliminary proposal to gain feedback from the LEWG.

Motivation

Modern C++ character types `char`, `char16_t`, and `char32_t` support Unicode Transformation Forms UTF-8, UTF-16, and UTF-32 respectively. Character type `wchar_t` also supports one of these encodings. Character and string literals and several forms of strings are supported for these character types. Use of more than one UTF encoding may appear in the same application, or even the same function. Yet neither the language nor the standard library provides a modern, convenient way to convert between these encodings. There is no equivalent to the ease with which the `std::to_string` family of functions can convert an arithmetic value to a string. This proposal solves the problems encountered by users due to the lack of convenient Unicode encoding conversions in the standard library. It does so in a way that meets the error handling requirements of the Unicode standard and the error handling needs requested by Unicode experts.

Example

Problem: Given a third-party function `f()` that returns a UTF-8 encoded `std::string` from a database, and a function `g()` from a different third-party that expects a UTF-16 encoded `std::u16string` as an argument, call `g()` with `f()` in a way that converts the string types and encodings, and handles errors according to the best practices documented in the Unicode standard.

Using the proposal:

```
string u8str(f()); // get a string that happens to be UTF-8 encoded
...
g(to_u16string(u8str)); // call a function that requires UTF-16
```

Without the proposal, using only the standard library: This might not be too difficult using a third-party library, but is surprisingly difficult using only the standard library. Unless the developer had enough Unicode experience to focus on error detection and to test against one of the existing UTF-8 test data sets, a roll-your-own solution would probably be very error-prone.

Prior proposal

[N3398](#), *String Interoperation Library*, proposed a complete overhaul of the standard library's mechanisms for character encoding conversion. The proposal was discussed at the Portland meeting in 2012. Some aspects of the proposal drew strong support, such as improving Unicode string interoperability. Other aspects drew strong opposition, such as new low level functionality to replace `std::codecvt`. Clearly participants did not want N3398 - they wanted a different proposal, less overreaching and more focused on Unicode encoding conversions. Bill Plauger summed it up when he said something like "Don't reinvent `codecvt`. That said, we should pick a winner — Unicode."

The current proposal is completely new and not a revision of N3398.

Implementation

A Boost licensed preliminary implementation is available at github.com/beman/unicode/tree/std-proposal.

Acknowledgements

Alisdair Meredith, Eric Niebler, Howard Hinnant, Jeffrey Yasskin, Marshall Clow, PJ Plauger, and Stephan T. Lavavej participated in the Portland discussion of N3398. Many of the design decisions that have gone into the current proposal flow directly from the Portland discussion.

After the Portland meeting, Matt Austern sat down with Google's "Unicode people" to "clarify things". His summary of that discussion was very helpful. Its guidance on error handling is reflected in current proposal.

Design decisions

Limit this proposal on UTF encoding conversion

- Getting UTF encoding conversion right is hard enough without also coping with other Unicode needs.
- No all-encompassing Unicode proposal is on the horizon.
- Other needs requested by domain experts, such as code point iterators, are important enough to deserve their own proposals.
- Guidance from both the committee and domain experts clearly favors a proposal focusing on UTF conversions.
- UTF encoding conversions can use character type alone to determine encoding, and this results in simpler conversion interfaces and no need to add a `char8_t` character type to the core language.

Provide three levels of functionality

Differing user needs is the primary motivation for providing three levels of functionality. Meshing well with existing standard library components such as STL algorithms and the `to_string` family of functions is an additional benefit.

- Provide high-level convenience encoding conversion functions that handle everyday string interoperability needs. Keep the interfaces simple. Example of use:

```
string u8str = u16str;
```

- Provide mid-level generic string conversion functions to support those requiring generic string interoperability needs. Example of use:

To Be supplied

- Provide a low-level encoding conversion algorithm patterned after existing standard library algorithms. Useful for users needing to perform encoding conversions on sequences of characters. Provides the underlying UTF encoding conversions for the other functions. Example of use:

To Be supplied

Provide a coherent error detection and handling policy

- Follow the Unicode standard's requirement that "A conformant encoding form conversion will treat any ill-formed code unit sequence as an error condition." (Unicode 3.9 D93 and C10).
- Provide a note to inform users of as to when they do and do not have to explicitly check for errors themselves.
- Provide error checking functions so that users can also explicitly check for errors to meet application needs.
- Handle errors via function objects to support the varied user needs described by domain experts.
- Default error handler function object follows the Unicode standard's recommendation of `U+FFFD` as a replacement character. Their rationale is that the other common approaches, including throwing an exception, can be and have been used as attack vectors.

Provide encoding conversions as explicitly called non-member functions

- Avoids possibly expensive hidden automatic encoding conversions in unexpected places.
- Avoids the need to change existing standard library components.
- Meshes well with the other `to_*string` functions already in the standard library.

Support `wchar_t` as well as `char`, `char16_t`, and `char32_t`

- `wchar_t` strings are the bridge to and from non-UTF encoded `char` strings, via existing standard library components using `codecvt` facets. This requires that `wchar_t` strings are UTF encoded, just as the proposal requires `char16_t` and `char32_t` strings be UTF encoded.

Place the proposed components in a unicode namespace

- Emphasizes that these functions assume char strings use a Unicode encoding.
- Signals that the committee cares about Unicode, but doesn't want to force it on users who prefer other encodings.
- Provides a home for these and future Unicode specific functions.

Keep interfaces neutral as to which character type or UTF encoding is "best"

- Each of these encodings have uses where it is preferred or required, and all of these needs may appear in the same application. For example:
 - UTF-8 is required by the API's for some operating systems.
 - UTF-16 is required to interfacing with existing databases that use UTF-16 encoding.
 - UTF-32 is preferred in code where every Unicode code point being encoded as a single code unit is advantageous.

Base conformance and definitions on the Unicode standard

- Repeating specifications that are already covered by the Unicode standard, and then trying to stay in sync as the Unicode standard evolves, is the path to insanity.
- In an "informative" (i.e. non-normative) section do describe some of the key Unicode definitions for the convenience of readers who have not memorized them.

Questions for the Library Evolution Working Group

- The specification for the to_*string convenience functions could be reduced from 16 signatures to four signatures by changing the argument types to SOURCE, and then specifying SOURCE as being any one of the four current argument types. The implementors could comply by supplying the full 16 signatures or by clever template metaprogramming. In other words, a less signatures versus more complex wording tradeoff. Does the LEWG/LWG have a strong preference either way?
- Should a second error handler that throws an exception be provided? Although the uffff error handler is clearly the best default, applications that work with supposedly well-formed UTF encodings may want an exception thrown if an ill-formed encoding is encountered.
- Should a new exception type such as encoding_error be provided?
- The Boost version will supply stream inserters and perhaps extractors that perform encoding conversion. Would LEWG/LWG like to see a similar proposal?

```
#include <boost/unicode/stream.hpp>
u16string str16(u"☺☺☺");
...
cout << str16 << '\n';
```

- The Boost version will supply codecvt-based encoding conversion between char and wchar_t strings. Would LEWG/LWG like to see a similar proposal?

```
#include <boost/unicode/codecvt_conversion.hpp>
string big5buf; // big-5 encoded
wstring wbuf; // UTF encoded
...
wbuf = codecvt_to_wstring(big5buf, big5_codecvt_facet);
```

To do

- If P0254, *Integrating std::string_view and std::string*, is accepted, then the proposed wording below needs to be reviewed to accommodate changes mandated by P0254. Such changes, if any, are expected to be minor.
- Add non-modifying sequence and string error-checking functions that detect ill formed encodings.
- Add arguments to error handler function objects:
 - The location of the error in the input sequence. Probably the iterator of the point where the error was detected.
 - The specifics of the error. Probably an error type enum for the specific encoding form errors called out by the Unicode Standard.

Proposed wording

Unicode library [unicode]

This clause describes components that C++ programs may use to perform operations on sequences and strings encoded in the Unicode character encoding forms UTF-32, UTF-16, and UTF-8.

Normative references [uni.refs]

The [Unicode Standard](#) is indispensable for the application of this document.^[footnote] The latest edition (including any amendments) applies. A reference to the Unicode Standard written in the form "(Unicode 3.4 D10)" refers to the Unicode Standard, Core Specification, chapter 3, section 4, clause D10.

^[Footnote] Unicode® is a registered trademark of Unicode, Inc. This information is given for the convenience of users of this document and does not constitute an endorsement by ISO or IEC of this product.

Conformance [uni.conf]

Any conflict between this Technical Specification's Unicode section ([unicode]) and the Unicode Standard, Chapter 3, C (conformance) and D (definitions) clauses is unintentional and should be resolved by reference to the Unicode Standard.

The normative definitions for the terms described informally in [uni.defs] are included in this Technical Specification by reference from the indicated D-clause definitions of the Unicode Standard.

Definitions (Informative) [uni.defs]

For convenience, informal summaries of definitions used in [unicode] are given here as quotes from the Unicode Standard.

Code point (Unicode 3.4 D10)

"Any value in the Unicode codespace. Informally, a code point can be thought of as a Unicode character."

(Unicode Appendix A - Notational Conventions):

"In running text, an individual Unicode code point is expressed as U+n, where n is four to six hexadecimal digits, using the digits 0–9 and uppercase letters A–F (for 10 through 15, respectively). Leading zeros are omitted, unless the code point would have fewer than four hexadecimal digits—for example, U+0001, U+0012, U+0123, U+1234, U+12345, U+102345.

[e.g.] U+0416 is the Unicode code point for the character named CYRILLIC CAPITAL LETTER ZHE."

Code unit (Unicode 3.9 D77)

"The minimal bit combination that can represent a unit of encoded text for processing or interchange. Code units are particular units of computer storage. ... The Unicode Standard uses 8-bit code units in the UTF-8 encoding form, 16-bit code units in the UTF-16 encoding form, and 32-bit code units in the UTF-32 encoding form."

[Note: In C++ one char, wchar_t, char16_t, or char32_t character holds one code unit. One to four code units (type char) are required to hold a UTF-8 encoded code point. One or two code units (type char16_t) are required to hold a UTF-16 encoded code point. One code unit (type char32_t) is required to hold a UTF-32 code point. Type wchar_t may use 8, 16, or 32-bit code units, encoded as UTF-8, UTF-16, or UTF-32, respectively, so will require 4, 2, or 1 code units to hold a code point depending on the encoding.—end note]

Unicode encoding forms (Unicode 3.9)

"The Unicode Standard supports three character encoding forms: UTF-32, UTF-16, and UTF-8. Each encoding form maps the Unicode code points U+0000..U+D7FF and U+E000..U+10FFFF to unique code unit sequences. The size of the code unit is specified for each encoding form. This section (Unicode 3.9) presents the formal definition of each of these encoding forms."

For formal definitions of UTF-32, UTF-16, and UTF-8, see [Section 3.9, Unicode Encoding Forms](#) in *The Unicode Standard*.

[Note: For general questions related to Unicode transformation form (UTF), UTF-8, UTF-16, UTF-32,

or byte order marks (BOM), see unicode.org/faq/utf_bom.html.—end note]

Well-formed (Unicode 3.9 D85)

"A Unicode code unit sequence that purports to be in a Unicode encoding form is called well-formed if and only if it does follow the specification of that Unicode encoding form."

Minimal well-formed code unit subsequence (Unicode 3.9 D85a)

"A well-formed Unicode code unit sequence that maps to a single Unicode scalar value.

- For UTF-8, see the specification in Unicode 3.9 D92 and Table 3-7.
- For UTF-16, see the specification in Unicode 3.9 D91.
- For UTF-32, see the specification in Unicode 3.9 D90."

Header <experimental/unicode> synopsis

```
namespace std {
namespace experimental {
inline namespace fundamentals_v2 {
namespace unicode {

    // Error function objects are called with no arguments and either throw an
    // exception or return a const pointer to a possibly empty C-style string.

    // default error handler: function object returns a C-string of type
    // ToCharT with a UTF encoded value of U+FFFD.

    // [uni.err], error handling
    template <class CharT> struct ufffd;
    template <> struct ufffd<char>;
    template <> struct ufffd<char16_t>;
    template <> struct ufffd<char32_t>;
    template <> struct ufffd<wchar_t>;

    // [uni.enc_cvt_alg], string encoding conversion algorithm
    template <class ToCharT, class InputIterator, class OutputIterator,
              class Error = typename ufffd<ToCharT>>
    OutputIterator convert_utf(InputIterator first, InputIterator last,
                              OutputIterator result, Error eh = Error());

    // [uni.gen_enc_cvt], string encoding generic conversions
    template <class ToCharT, class FromCharT,
              class FromTraits = typename char_traits<FromCharT>,
              class View = basic_string_view<FromCharT, FromTraits>,
              class Error = ufffd<ToCharT>,
              class ToTraits = char_traits<ToCharT>,
              class ToAlloc = allocator<ToCharT>>
    basic_string<ToCharT, ToTraits, ToAlloc>
    to_utf_string(View v, Error eh = Error(), const ToAlloc& a = ToAlloc());

    // [uni.conv_enc_cvt], string encoding convenience conversions
    template <class Error = ufffd<char>>
    string to_u8string(string_view v, Error eh = Error());
    template <class Error = ufffd<char>>
    string to_u8string(u16string_view v, Error eh = Error());
    template <class Error = ufffd<char>>
    string to_u8string(u32string_view v, Error eh = Error());
    template <class Error = ufffd<char>>
    string to_u8string(wstring_view v, Error eh = Error());

    template <class Error = ufffd<char16_t>>
    u16string to_u16string(string_view v, Error eh = Error());
    template <class Error = ufffd<char16_t>>
    u16string to_u16string(u16string_view v, Error eh = Error());
    template <class Error = ufffd<char16_t>>
    u16string to_u16string(u32string_view v, Error eh = Error());
    template <class Error = ufffd<char16_t>>
    u16string to_u16string(wstring_view v, Error eh = Error());

    template <class Error = ufffd<char32_t>>
    u32string to_u32string(string_view v, Error eh = Error());
    template <class Error = ufffd<char32_t>>
    u32string to_u32string(u16string_view v, Error eh = Error());
    template <class Error = ufffd<char32_t>>
    u32string to_u32string(u32string_view v, Error eh = Error());
    template <class Error = ufffd<char32_t>>
    u32string to_u32string(wstring_view v, Error eh = Error());
```

```

template <class Error = uffffd<wchar_t>>
wstring to_wstring(string_view v, Error eh = Error());
template <class Error = uffffd<wchar_t>>
wstring to_wstring(u16string_view v, Error eh = Error());
template <class Error = uffffd<wchar_t>>
wstring to_wstring(u32string_view v, Error eh = Error());
template <class Error = uffffd<wchar_t>>
wstring to_wstring(wstring_view v, Error eh = Error());

} // namespace unicode
} // namespace fundamentals_v2
} // namespace experimental
} // namespace std

```

UTF encoding conversion functions [uni.enc_cvt]

UTF conversion functions determine encoding based on character type. The relationship between character type and encoding is specified by the following table:

<i>UTF Conversions</i>	
Character Type	Encoding
char	UTF-8
char16_t	UTF-16
char32_t	UTF-32
wchar_t	UTF-8, 16, or 32

Error handling [uni.err]

When an ill-formed code unit subsequence is detected during execution of a conversion function, an *error handler* function object is invoked. Unless the error handler throws an exception, the string returned by the error handler is added to the output sequence and the ill-formed input code unit subsequence is not added to the output sequence. Detection of ill-formed code unit subsequences is required even when the input and output encodings are the same. *[Note: If the error handler function object always returns a well-formed UTF character sequence, the conversions function's entire output sequence is a well-formed UTF sequence. — end note]*

```

template <class CharT> struct uffffd;
template <> struct uffffd<char>;
template <> struct uffffd<char16_t>;
template <> struct uffffd<char32_t>;
template <> struct uffffd<wchar_t>;

```

struct uffffd provides the default error handler function object for conversion functions. The default error handling function object returns U+FFFD REPLACEMENT CHARACTER as a single code point error marker. Each specialization shall provide a member function with the signature:

```
constexpr CharT* operator()() const noexcept;
```

that returns the value indicated in the Specializations table:

<i>Specializations</i>	
CharT	Returns
char	u8"\uFFFD"
char16_t	u"\uFFFD"
char32_t	U"\uFFFD"
wchar_t	L"\uFFFD"

[Note: U+FFFD REPLACEMENT CHARACTER is returned as the default single code point error marker in accordance with the recommendations of the Unicode Standard. The rationale given by the Unicode standard is essentially that other commonly used approaches, including throwing exceptions, can be and have been used as security attack vectors. —end note]

Encoding conversion algorithm [uni.enc_cvt_alg]

```

template <class ToCharT, class InputIterator, class OutputIterator,
class Error = typename uffffd<ToCharT>>
OutputIterator convert_utf(InputIterator first, InputIterator last,
OutputIterator result, Error eh = Error());

```

Effects: For each minimal well-formed or ill-formed code unit subsequence in the range [first, last):

- If the code unit subsequence is well-formed, copies the subsequence's Unicode scalar value by performing `*result++ = *u++` where `u` is a `ToCharT*` pointing to the code units required to represent the subsequence's Unicode scalar value in the encoding form of result.
- Otherwise, copies the null-terminated string returned by the `eh` function object by performing `*result++ = *p++` for each successive value of a pointer `p` to the returned string.

Returns: result.

Remarks: The Unicode encoding form for the range [first, last) is determined by `InputIterator` value type ([`uni.enc_cvt`]). The Unicode encoding form for result is determined by `ToCharT` ([`uni.enc_cvt`]).

Generic string encoding conversion functions [`uni.gen_enc_cvt`]

```
template <class ToCharT, class FromCharT,
class FromTraits = typename char_traits<FromCharT>,
class View = basic_string_view<FromCharT, FromTraits>,
class Error = ufffd<ToCharT>,
class ToTraits = char_traits<ToCharT>,
class ToAlloc = allocator<ToCharT>>
basic_string<ToCharT, ToTraits, ToAlloc>
to_utf_string(View v, Error eh = Error(), const ToAlloc& a = ToAlloc());
```

Returns: Equivalent to:

```
basic_string<ToCharT, ToTraits, ToAlloc> tmp(a);
convert_utf<ToCharT>(v.cbegin(), v.cend(), back_inserter(tmp), eh);
return tmp;
```

Convenience string encoding conversion functions [`uni.conv_enc_cvt`]

```
template <class Error = ufffd<char>>
string to_u8string(string_view v, Error eh = Error());
template <class Error = ufffd<char>>
string to_u8string(u16string_view v, Error eh = Error());
template <class Error = ufffd<char>>
string to_u8string(u32string_view v, Error eh = Error());
template <class Error = ufffd<char>>
string to_u8string(wstring_view v, Error eh = Error());

template <class Error = ufffd<char16_t>>
u16string to_u16string(string_view v, Error eh = Error());
template <class Error = ufffd<char16_t>>
u16string to_u16string(u16string_view v, Error eh = Error());
template <class Error = ufffd<char16_t>>
u16string to_u16string(u32string_view v, Error eh = Error());
template <class Error = ufffd<char16_t>>
u16string to_u16string(wstring_view v, Error eh = Error());

template <class Error = ufffd<char32_t>>
u32string to_u32string(string_view v, Error eh = Error());
template <class Error = ufffd<char32_t>>
u32string to_u32string(u16string_view v, Error eh = Error());
template <class Error = ufffd<char32_t>>
u32string to_u32string(u32string_view v, Error eh = Error());
template <class Error = ufffd<char32_t>>
u32string to_u32string(wstring_view v, Error eh = Error());

template <class Error = ufffd<wchar_t>>
wstring to_wstring(string_view v, Error eh = Error());
template <class Error = ufffd<wchar_t>>
wstring to_wstring(u16string_view v, Error eh = Error());
template <class Error = ufffd<wchar_t>>
wstring to_wstring(u32string_view v, Error eh = Error());
template <class Error = ufffd<wchar_t>>
wstring to_wstring(wstring_view v, Error eh = Error());
```

Returns: Equivalent to:

```
to_utf_string<r_value_type, v_value_type, Error>(v, eh) where r_value_type
is the value_type of the basic_string to be returned and v_value_type is
the value_type of v.
```

